

Step 2: Database Persistence Layer

JDBC, PostgreSQL Integration, & Object-Relational Mapping with JPA / Hibernate

Course: Advanced Java Enterprise Engineering (EILMS Project) | Module: Data Persistence & Database Architecture

1. Executive Summary & Architectural Context

In Step 1 of the Enterprise Inventory & Logistics Management System (EILMS), we established an in-memory, thread-safe domain model. While in-memory processing provides low latency, state is volatile and lost upon application shutdown. Step 2 shifts our application architecture from transient state management to **durable relational data persistence** using **PostgreSQL**.

This module introduces a tiered persistence strategy: beginning with direct low-level **JDBC (Java Database Connectivity)** for baseline connection management, and escalating to enterprise-grade **JPA (Jakarta Persistence API)** and **Hibernate ORM** for declarative object-relational mapping.

2. Maven Configuration & Dependency Integration

To enable database connectivity and ORM capabilities in our Maven project (`eilms-core`), we must append the PostgreSQL database driver, Hibernate ORM core, and HikariCP connection pool dependencies into `pom.xml`.

```
<!-- pom.xml Dependencies Fragment -->
<dependencies>
  <!-- PostgreSQL JDBC Driver -->
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.2</version>
  </dependency>

  <!-- Jakarta Persistence API & Hibernate ORM Core -->
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.4.4.Final</version>
  </dependency>

  <!-- HikariCP Connection Pool -->
  <dependency>
    <groupId>com.zaxxer</groupId>
    <artifactId>HikariCP</artifactId>
    <version>5.1.0</version>
  </dependency>
</dependencies>
```

PROFESSOR'S PEDAGOGICAL ANALYSIS: DEPENDENCY SELECTION

- **PostgreSQL JDBC Driver:** Implements Java's `java.sql.Driver` interface to translate JDBC calls into PostgreSQL's wire protocol over socket connections (port 5432).
- **Hibernate ORM Core:** Serves as the concrete reference implementation of the Jakarta Persistence API standard, handling standard SQL generation, entity state transitions, and first-level session caching.
- **HikariCP:** Standard high-performance JDBC connection pool that maintains pre-allocated physical socket connections to PostgreSQL, avoiding expensive TCP connection handshakes per transaction.

3. Implementation Details & Pedagogical Code Walkthrough

File 1: `src/main/java/com/eilms/config/DatabaseConfig.java` (JDBC Baseline)

Before abstracting to ORM frameworks, engineers must understand raw database connectivity and statement execution safety.

```
package com.eilms.config;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConfig {
    private static final String URL = "jdbc:postgresql://localhost:5432/eilms_db";
    private static final String USER = "postgres";
    private static final String PASSWORD = "postgres_password";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

PROFESSOR'S EXPLANATION: WHY WE CREATED THIS CODE

- **Baseline Verification:** Provides a lightweight mechanism to test direct network socket connectivity to the PostgreSQL server without framework overhead.
- **DriverManager Role:** Uses `DriverManager` to negotiate database sessions. In enterprise production, this class is wrapped by a managed `DataSource` pool.

File 2: `src/main/java/com/eilms/entity/WarehouseItemEntity.java` (JPA Domain Model)

To map domain objects directly to relational tables, we create a specialized persistent entity annotated with JPA metadata.

```
package com.eilms.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "warehouse_items")
public class WarehouseItemEntity {

    @Id
    @Column(name = "item_id", length = 50)
    private String id;

    @Column(name = "item_name", nullable = false, length = 150)
    private String name;

    @Column(name = "stock_quantity", nullable = false)
    private int quantity;

    // Default No-Arg Constructor required by JPA reflection runtime
    public WarehouseItemEntity() {}

    public WarehouseItemEntity(String id, String name, int quantity) {
        this.id = id;
        this.name = name;
        this.quantity = quantity;
    }
}
```

```
// Getters and Setters
public String getId() { return id; }
public void setId(String id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public int getQuantity() { return quantity; }
public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

PROFESSOR'S EXPLANATION: WHY WE CREATED THIS CODE

- **JPA Metadata Annotations:** `@Entity` marks the class as a persistent database entity; `@Table(name = "warehouse_items")` explicitly maps it to the relational target table.
- **Primary Key Specification:** `@Id` designates `id` as the primary key column for database identity resolution.
- **No-Arg Constructor Constraint:** JPA providers use Java Reflection to instantiate entity instances dynamically when converting SQL `ResultSet` rows into Java objects. A public or protected default constructor is strictly required.

File 3: `src/main/resources/META-INF/persistence.xml` (JPA Configuration)

The standard JPA deployment descriptor defines persistence units, database connection properties, and ORM dialect instructions.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/
persistence/persistence_3_0.xsd"
  version="3.0">

  <persistence-unit name="eilms-pu">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>com.eilms.entity.WarehouseItemEntity</class>

    <properties>
      <!-- Database Driver and Connection Credentials -->
      <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver"/>
      <property name="jakarta.persistence.jdbc.url" value="jdbc:postgresql://localhost:5432/
eilms_db"/>
      <property name="jakarta.persistence.jdbc.user" value="postgres"/>
      <property name="jakarta.persistence.jdbc.password" value="postgres_password"/>

      <!-- Dialect and DDL Auto-Generation Settings -->
      <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect"/>
      <property name="hibernate.hbm2ddl.auto" value="update"/>
      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.format_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

PROFESSOR'S EXPLANATION: WHY WE CREATED THIS CODE

- **PostgreSQL Dialect:** Informs Hibernate to generate SQL syntax specific to PostgreSQL (e.g., handling specific column types, sequence generators, and transactional locking mechanisms).

- **Schema Auto-Generation** ('`hbm2ddl.auto`'): Setting this to `update` instructs Hibernate to compare annotated entity definitions against existing PostgreSQL metadata and auto-generate or alter DDL schemas automatically upon application start.

File 4: `src/main/java/com/eilms/PersistenceTestMain.java` (Execution Engine)

A bootstrap class executing ACID-compliant entity transactions via `EntityManager`.

```
package com.eilms;

import com.eilms.entity.WarehouseItemEntity;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class PersistenceTestMain {
    public static void main(String[] args) {
        // 1. Build EntityManagerFactory for the defined persistence unit
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("eilms-pu");

        // 2. Obtain an active EntityManager instance
        EntityManager em = emf.createEntityManager();

        // 3. Begin Transaction
        em.getTransaction().begin();

        // 4. Instantiate and Persist Domain Entity
        WarehouseItemEntity item = new WarehouseItemEntity("ITEM-201", "PostgreSQL Server Node", 25);
        em.persist(item);

        // 5. Commit Transaction to write changes to PostgreSQL
        em.getTransaction().commit();

        System.out.println(">>> Entity successfully persisted to PostgreSQL database!");

        // 6. Resource Cleanup
        em.close();
        emf.close();
    }
}
```

PROFESSOR'S EXPLANATION: WHY WE CREATED THIS CODE

- **EntityManagerFactory:** Heavyweight, thread-safe factory object that parses `persistence.xml` and builds metadata models. Created once per application lifecycle.
- **EntityManager Context:** Lightweight, non-thread-safe persistence context that manages entity state (Transient, Managed, Detached, Removed).
- **Explicit Transaction Boundaries:** Relational modifications must take place inside explicit `begin()` and `commit()` boundaries to guarantee database ACID guarantees.

4. Execution & PostgreSQL Verification

Executing `PersistenceTestMain.java` in Eclipse produces formatted SQL output in the console and establishes the underlying relational structure inside PostgreSQL:

```
Hibernate:
create table warehouse_items (
    stock_quantity integer not null,
```

```
        item_id varchar(50) not null,  
        item_name varchar(150) not null,  
        primary key (item_id)  
    )  
Hibernate:  
    insert  
    into  
        warehouse_items  
        (item_name, stock_quantity, item_id)  
    values  
        (?, ?, ?)  
>>> Entity successfully persisted to PostgreSQL database!
```

item_id (PK)	item_name	stock_quantity
ITEM-201	PostgreSQL Server Node	25

End of Architectural Lecture Guide — Step 2 Data Persistence Layer